SNES Assembly Adventure 10: Modular Programming

21 Apr 2022

Welcome back, Adventurer! Last time, we learned how to build our code with CMake.

This time, we'll restructure our sprite demo to

- separate assets into their own subdirectory
- autogenerate assets and headers with CMake

At the end of this tutorial, you'll have a nice project template you can reuse in the future. As always, the code can be found on GitHub.



Without further ado, let's get started!

Keep Assets Separate From Code

We already saw a bit of code generation with CMake in the last tutorial. At the moment, our source and asset files live in the same directory. As your project grows, this can become unwieldy. So the first thing we'll do is move the assets into their own subdirectory. Create a new subdirectory `assets` and move both `SpriteColors.pal` and `Sprites.vra` into it. Create a new file called `CMakeLists.txt` in the new subdirectory `assets`:

```
# snes_cradle/assets/CMakeLists.txt

cmake_minimum_required( VERSION 3.21 FATAL_ERROR )
```

We'll add commands to it in a moment. Your directory structure should look like this now:

```
snes_cradle/
    assets/
        CMakeLists.txt
        SpriteColors.pal
        Sprites.vra
    cmake/
        CMakeCA65816Compiler.cmake.in
        CMakeCA65816Information.cmake
        CMakeDetermineCA65816Compiler.cmake
        CMakeTestCA65816Compiler.cmake
    src/
        CMakeLists.txt
        JoypadSprite.s
    CMakeLists.txt
    MemoryMap.cfg
```

If we try to build this, it will fail with the following error messages:

```
$ cmake --build build
[1/2] Building CA65816 object CMakeFiles\JoypadSpriteDemo.dir\src\Joypa
FAILED: CMakeFiles/JoypadSpriteDemo.dir/src/JoypadSprite.s.o
ca65.exe --cpu 65816
```

```
          -s
          -o CMakeFiles\JoypadSpriteDemo.dir\src\JoypadSprite.s.o
          ~\snes_cradle\src\JoypadSprite.s
~\snes_cradle\src\JoypadSprite.s:69: Error: Cannot open include file 'S
~\snes_cradle\src\JoypadSprite.s:70: Error: Cannot open include file 'S
ninja: build stopped: subcommand failed.
```

As you can see, ca65 can no longer find `Sprites.vra` and `SpriteColors.pal` after we moved them into `assets` (The output has been edited for clarity, so your output may differ a bit). This problem can be resolved in two ways. We could change the commands in `.incbin "../assets/Sprites.vra"` and `.incbin "../assets/SpriteColors.pal"`, respectively. The assembler will then find the asset files with no problem. But this isn't ideal. As the number of source and asset files grows, we'll probably want to move them into a more granular directory structure. So each time we move a file, we would have to change every single file that references it. That's how bugs and build errors are born.

The second option is to tell the ca65 assembler where to find include files. The documentation tells us that the `--bin-include-dir` command line argument is used for that. So we need to execute

```
$ ca65 --cpu 65816 --bin-include-dir ../assets -s -o JoypadSprite.o Joy
```

For the source code to assemble correctly. This puts us a bit in a pickle. The ca65 assembler uses two different command line arguments for file inclusion: one for (binary) files included with `.incbin` (`--bin-include-dir`), and another for `.include` (`--include-dir` or shorter `-I`).

The question we have to answer here is whether we want to use the CMake command `target_include_directories` for binary (`.incbin`) or source files (`.include`). I've decided to opt for the latter, as a SNES game project will probably include a bit more source files than binary/asset files.

For binary files, we'll instead use the CMake command `target_compile_options` to add `--bin-include-dir` to our build command. Let's edit `assets/CMakeLists.txt`:

```
# snes_cradle/assets/CMakeLists.txt

cmake_minimum_required( VERSION 3.21 FATAL_ERROR )

target_compile_options( ${PROJECT_NAME}
    PRIVATE "SHELL:--bin-include-dir ${CMAKE_CURRENT_SOURCE_DIR}" )
```

Here we tell CMake to add `--bin-include-dir` followed by the directory the `CMakeLists.txt` file is located in (here, `assets`) to the compiler flags for the target. Next, we need to tell CMake what to do with those flags. The `SHELL` part is necessary to avoid option de-duplication. See ca65 search paths for details.

Next, we need to add our new `assets` subdirectory to the main `CMakeLists.txt` in the project's root directory file:

```
# snes_cradle/CMakeLists.txt

cmake_minimum_required( VERSION 3.21 FATAL_ERROR )

set( MEMORY_MAP_FILE ${CMAKE_CURRENT_SOURCE_DIR}/MemoryMap.cfg )
list( APPEND CMAKE_MODULE_PATH "${CMAKE_SOURCE_DIR}/cmake" )

project( JoypadSpriteDemo
         LANGUAGES CA65816 )

add_executable( ${PROJECT_NAME} "" )
add_subdirectory( assets )
add_subdirectory( src )
set_target_properties( ${PROJECT_NAME}
    PROPERTIES SUFFIX ".smc" )

# Check if the path to the memory map file is set
if( NOT DEFINED MEMORY_MAP_FILE )
  message( FATAL_ERROR "Path to memory map file not set!" )
endif( NOT DEFINED MEMORY_MAP_FILE )
```

All we did here was add `add_subdirectory( assets )`. Finally, we need to update the build command itself in `cmake/CMakeCA65816Information.cmake`:

```
# snes_cradle/cmake/CMakeCA65816Information.cmake

# How to build objects
set( CMAKE_CA65816_COMPILE_OBJECT
    "<CMAKE_CA65816_COMPILER> --cpu 65816 \
                              <FLAGS> \
                              -s \
                              -o <OBJECT> \
                              <SOURCE>"
)

# How to build executables
set( CMAKE_CA65816_LINK_EXECUTABLE
    "ld65 -C ${CMAKE_SOURCE_DIR}/MemoryMap.cfg \
          <OBJECTS> \
          -o <TARGET>"
)

set( CMAKE_CA65816_INFORMATION_LOADED 1 )
```

We only need to add a single line again, namely, the `<FLAGS>` argument. Now rerun CMake and the error message from earlier should no longer show. As always, test the ROM you just built in your emulator and verify everything still works.

Creating a Header and Source File For Assets

We'll need to generate a header so that different source files can access the addresses where the assets are stored. We want to be able to access assets/addresses with the `.include` directive. In contrast, if we included the assets with `.incbin` in each source file (that accesses the assets) it could copy the whole asset data several times into the ROM! That'd be a waste of precious ROM memory. Instead, we're going to create a header and a source file for the assets. Thankfully, cc65 makes it easy for us to create something akin to header guards.

First, we're going to create a new source file called `assets/Assets.s`:

```
; ------------------------------------------------------------------------
;   File: Assets.s
;   Description: Creates a segment for all assets and exports symbols to make
;   them accessible to other parts of the project.
; ------------------------------------------------------------------------

;----- Export ------------------------------------------------------------
.export     SpriteData
.export     ColorData
;-------------------------------------------------------------------------

;----- Assset Data -------------------------------------------------------
.segment "SPRITEDATA"
SpriteData: .incbin "Sprites.vra"
ColorData:  .incbin "SpriteColors.pal"
;-------------------------------------------------------------------------
```

Assets.s hosted with ❤ by **GitHub** **view raw**

This file essentially moves the `SPRITEDATA` segment from lines 68 through 70 of `src/JoypadSprite.s` into its own file. It also adds a couple of `.export` directives to make the symbols available to other parts of the code (i.e., modules). In general, all symbols are local to the files/modules they're in. If you want to make them available to other sources/modules, you need to export them.

Next, we create the `assets/Assets.inc` header file:

```
.ifndef ASSETS_INC
ASSETS_INC = 1
; ------------------------------------------------------------------------
;   File: Assets.inc
;   Description: Holds address symbols for all data defined in Assets.s
; ------------------------------------------------------------------------

;----- Import ------------------------------------------------------------
.import     SpriteData
.import     ColorData
;-------------------------------------------------------------------------
.endif
```

Assets.inc hosted with ❤ by **GitHub** **view raw**

This file simply imports the symbols we just exported from `assets/Assets.s` into each file that includes `assets/Asset.inc`. We add a header guard to protect the file from multiple inclusions (and therefore causing symbol redefined errors).

Since there's a new file that needs assembling, we also update `assets/CMakeLists.txt`:

```
# snes_cradle/assets/CMakeLists.txt

cmake_minimum_required( VERSION 3.21 FATAL_ERROR )

target_compile_options( ${PROJECT_NAME}
    PRIVATE "SHELL:-I ${CMAKE_CURRENT_SOURCE_DIR}" )

target_sources( ${PROJECT_NAME}
    PRIVATE Assets.s )
```

Note that we replaced some options in `target_compile_options`. This is about `--bin-include-dir` VS `--include-dir` we talked about above.

Finally, we update `src/JoypadSprite.s`. Just replace lines 68 through 70 with `.include "Assets.inc". Don't forget to rerun CMake and test your ROM in an emulator.

Your directory structure should now look like this:

```
snes_cradle/
    assets/
        Assets.inc
        Assets.s
        CMakeLists.txt
        SpriteColors.pal
        Sprites.vra
    cmake/
        CMakeCA65816Compiler.cmake.in
        CMakeCA65816Information.cmake
        CMakeDetermineCA65816Compiler.cmake
        CMakeTestCA65816Compiler.cmake
    src/
        CMakeLists.txt
```

```
    JoypadSprite.s
CMakeLists.txt
MemoryMap.cfg
```

I chose to put all assets into a single header for convenience. But you can be as granular as you want. For example, it would probably be useful to be able to access your main character's and enemy graphics separately. Just create a separate header for each (whether the graphic data itself goes into a separate module, depends on your ROM's memory layout).

Refactoring `JoypadSprite.s`

Finally, we're going to split up our main source file into smaller chunks. Several parts of the main source make use of the constants defined at the beginning of `src/JoypadSprite.s`. So let's split those off for easy access. Create a new subdirectory `src/common` with three new header files in it:

```
.ifndef GAME_CONSTANTS_INC
GAME_CONSTANTS_INC = 1
; ------------------------------------------------------------------------------
;   File: GameConstants.inc
;   Description: Constant values used by this demo
; ------------------------------------------------------------------------------

;----- Game Constants --------------------------------------------------------
    ; we use these constants to check for collisions with the screen boundaries
SCREEN_LEFT     = $00   ; left screen boundary = 0
SCREEN_RIGHT    = $ff   ; right screen boundary = 255
SCREEN_TOP      = $00   ; top screen boundary = 0
SCREEN_BOTTOM   = $df   ; bottom screen boundary = 223
    ; a simple constant to define the sprite movement speed
SPRITE_SPEED    = $02   ; the sprites will move 2 pixel per frame
    ; this makes the code a bit more readable
SPRITE_SIZE     = $08   ; sprites are 8 by 8 pixel
OAMMIRROR_SIZE  = $0220 ; OAMRAM can hold data for 128 sprites, 4 bytes each
    ; constants to use as masks
UP_BUTTON       = $0800
DOWN_BUTTON     = $0400
LEFT_BUTTON     = $0200
RIGHT_BUTTON    = $0100
;------------------------------------------------------------------------------

```

GameConstants.inc hosted with ❤ by **GitHub** **view raw**

```
.ifndef MEMORY_MAP_WRAM_INC
MEMORY_MAP_WRAM_INC = 1
; ------------------------------------------------------------------------------
;   File: MemoryMapWRAM.inc
;   Description: Symbols representing addresses in WRAM
; ------------------------------------------------------------------------------

;----- Memory Map WRAM ---------------------------------------------------------
HOR_SPEED   = $0300     ; the horizontal speed
VER_SPEED   = $0301     ; the vertical speed
JOYPAD1     = $0302     ; data read from joypad 1
JOYTRIGGER1 = $0304     ; trigger read from joypad 1
JOYHELD1    = $0306     ; held buttons read from joypad 1
OAMMIRROR   = $0400     ; location of OAMRAM mirror in WRAM
;------------------------------------------------------------------------------

.endif
```

MemoryMapWRAM.inc hosted with ❤ by **GitHub** **view raw**

```
.ifndef REGISTERS_INC
REGISTERS_INC = 1
; ------------------------------------------------------------------------------
;   File: Registers.inc
;   Description: Symbols representing the memory mapped registers of the SNES
; ------------------------------------------------------------------------------

;----- Aliases/Labels ----------------------------------------------------------
; these are aliases for the Memory Mapped Registers we will use
INIDISP     = $2100     ; inital settings for screen
OBJSEL      = $2101     ; object size $ object data area designation
OAMADDL     = $2102     ; address for accessing OAM
OAMADDH     = $2103
OAMDATA     = $2104     ; data for OAM write
VMAINC      = $2115     ; VRAM address increment value designation
VMADDL      = $2116     ; address for VRAM read and write
VMADDH      = $2117
VMDATAL     = $2118     ; data for VRAM write
VMDATAH     = $2119
CGADD       = $2121     ; address for CGRAM read and write
CGDATA      = $2122     ; data for CGRAM write
```

```
22  TM          = $212c     ; main screen designation
23  HVBJOY      = $4212     ; H/V blank flags and standard controller enable flag
24  STDCNTRL1L  = $4218     ; data for standard controller I
25  STDCNTRL1H  = $4219
26  NMITIMEN    = $4200     ; enable flaog for v-blank
27  MDMAEN      = $420b     ; DMA enable register
28  RDNMI       = $4210     ; read the NMI flag status
29  DMAP0       = $4300     ; DMA control register, channel 0
30  BBAD0       = $4301     ; DMA destination register, channel 0
31  A1T0L       = $4302     ; DMA source address register low, channel 0
32  A1T0H       = $4303     ; DMA source address register high, channel 0
33  A1T0B       = $4304     ; DMA source address register bank, channel 0
34  DAS0L       = $4305     ; DMA size register low, channel 0
35  DAS0H       = $4306     ; DMA size register high, channel 0
36  ;---------------------------------------------------------------------------
37
```

Registers.inc hosted with ❤ by **GitHub** **view raw**

As with the assets, you can get as granular with your headers as you want. I personally tend to keep my code in many separate files. As your code grows, it is a lot easier to merge several files into one later on than to split up code (several modules/subroutines/etc.)

As always, we also need a new `src/common/CMakeLists.txt`:

```
# snes_cradle/src/common/CMakeLists.txt

cmake_minimum_required( VERSION 3.21 FATAL_ERROR )

target_compile_options( ${PROJECT_NAME}
    PRIVATE "SHELL:-I ${CMAKE_CURRENT_SOURCE_DIR}" )
```

Modify `src/CMakeLists.txt` to include `add_subdirectory( common )`. Remove lines 6 through 62 in `src/JoypadSprite.s` and include the newly created headers with `.include`.

Now we're going to factor out some of the subroutines related to graphics. Namely, `LoadVRAM`, `LoadCGRAM`, and `UpdateOAMRAM`. Those functions are universal, you can use them in all kinds of projects. I choose to put them in a subdirectory called `src/PPU`, where I keep all code related to the PPU/graphics.

We're going to follow a similar pattern as we did with the assets. Create a new header-source pair `src/PPU/PPU.s` and `src/PPU/PPU.inc` respectively:

```
.ifndef PPU_INC
PPU_INC = 1
; ------------------------------------------------------------------------------
;   File: PPU.inc
;   Description: Holds address symbols for all subroutines defined in PPU.s
; ------------------------------------------------------------------------------

;----- Import ----------------------------------------------------------------
.import     LoadVRAM
.import     LoadCGRAM
.import     UpdateOAMRAM
;------------------------------------------------------------------------------
.endif
```

PPU.inc hosted with ❤ by **GitHub** **view raw**

```
; ------------------------------------------------------------------------------
;   File: PPU.s
;   Description: A collection of subroutines for interacting with VRAM, CGRAM,
;   OAMRAM, and the PPU.
; ------------------------------------------------------------------------------

;----- Export ----------------------------------------------------------------
.export     LoadVRAM
.export     LoadCGRAM
.export     UpdateOAMRAM
;------------------------------------------------------------------------------

;----- Assembler Directives --------------------------------------------------
.p816                           ; tell the assembler this is 65816 code
.A8                             ; set accumulator to 8-bit
.I16                            ; set index registers to 16-bit
;------------------------------------------------------------------------------

;----- Includes --------------------------------------------------------------
.include "Registers.inc"
;------------------------------------------------------------------------------

.segment "CODE"
;------------------------------------------------------------------------------
;   Load sprite data into VRAM
```

```
26  ;   Parameters: NumBytes: .byte, SrcPointer: .addr, DestPointer: .addr
27  ;----------------------------------------------------------------------------
28  .proc   LoadVRAM
29          phx                     ; save old stack pointer
30          ; create frame pointer
31          phd                     ; push Direct Register to stack
32          tsc                     ; transfer Stack to... (via Accumulator)
33          tcd                     ; ...Direct Register.
34          ; use constants to access arguments on stack with Direct Addressing
35          NumBytes    = $07       ; number of bytes to transfer
36          SrcPointer  = $08       ; source address of sprite data
37          DestPointer = $0a       ; destination address in VRAM
38
39          ; set destination address in VRAM, and address increment after writing
40          ldx DestPointer         ; load the destination pointer...
41          stx VMADDL              ; ...and set VRAM address register to it
42          lda #$80
43          sta VMAINC              ; increment VRAM address by 1 when writing to
44
45          ; loop through source data and transfer to VRAM
46          ldy #$0000              ; set register Y to zero, we will use Y as a l
47  VRAMLoop:
48          lda (SrcPointer, S), Y  ; get bitplane 0/2 byte from the sprite data
49          sta VMDATAL             ; write the byte in A to VRAM
50          iny                     ; increment counter/offset
51          lda (SrcPointer, S), Y  ; get bitplane 1/3 byte from the sprite data
52          sta VMDATAH             ; write the byte in A to VRAM
53          iny                     ; increment counter/offset
54          cpy NumBytes            ; check whether we have written $04 * $20 = $8
55          bcc VRAMLoop            ; if X is smaller than $80, continue the loop
56
57          ; all done
58          pld                     ; restore caller's frame pointer
59          plx                     ; restore old stack pointer
60          rts
61  .endproc
62  ;----------------------------------------------------------------------------
63
64  ;----------------------------------------------------------------------------
65  ;   Load color data into CGRAM
66  ;   NumBytes: .byte, SrcPointer: .byte, DestPointer: .addr
67  ;----------------------------------------------------------------------------
68  .proc   LoadCGRAM
69          phx                     ; save old stack pointer
```

```
70          ; create frame pointer
71          phd                     ; push Direct Register to stack
72          tsc                     ; transfer Stack to... (via Accumulator)
73          tcd                     ; ...Direct Register.
74          ; use constants to access arguments on stack with Direct Addressing
75          NumBytes    = $07       ; number of bytes to transfer
76          SrcPointer  = $08       ; source address of sprite data
77          DestPointer = $0a       ; destination address in VRAM
78
79          ; set CGDRAM destination address
80          lda DestPointer         ; get destination address
81          sta CGADD               ; set CGRAM destination address
82
83          ldy #$0000              ; set Y to zero, use it as loop counter and of
84  CGRAMLoop:
85          lda (SrcPointer, S), Y  ; get the color low byte
86          sta CGDATA              ; store it in CGRAM
87          iny                     ; increase counter/offset
88          lda (SrcPointer, S), Y  ; get the color high byte
89          sta CGDATA              ; store it in CGRAM
90          iny                     ; increase counter/offset
91          cpy NumBytes            ; check whether 32/$20 bytes were transfered
92          bcc CGRAMLoop           ; if not, continue loop
93
94          ; all done
95          pld                     ; restore caller's frame pointer
96          plx                     ; restore old stack pointer
97          rts
98  .endproc
99  ;-------------------------------------------------------------------------
100
101 ;-------------------------------------------------------------------------
102 ;   Copies the OAMRAM mirror into OAMRAM
103 ;-------------------------------------------------------------------------
104 .proc   UpdateOAMRAM
105         phx                     ; save old stack pointer
106         ; create frame pointer
107         phd                     ; push Direct Register to stack
108         tsc                     ; transfer Stack to... (via Accumulator)
109         tcd                     ; ...Direct Register.
110         ; use constants to access arguments on stack with Direct Addressing
111         MirrorAddr  = $07       ; address of the mirror we want to copy
112
113         ; set up DMA channel 0 to transfer data to OAMRAM
```

```
114         lda #%00000010          ; set DMA channel 0
115         sta DMAP0
116         lda #$04                ; set destination to OAMDATA
117         sta BBAD0
118         ldx MirrorAddr          ; get address of OAMRAM mirror
119         stx A1T0L               ; set low and high byte of address
120         stz A1T0B               ; set bank to zero, since the mirror is in WRA
121         ldx #$0220              ; set the number of bytes to transfer
122         stx DAS0L
123
124         lda #$01                ; start DMA transfer
125         sta MDMAEN
126
127         ; OAMRAM update is done, restore frame and stack pointer
128         pld                     ; restore caller's frame pointer
129         plx                     ; restore old stack pointer
130         rts
131 .endproc
132 ;---------------------------------------------------------------------------
```

PPU.s hosted with ❤ by **GitHub** **view raw**

Same idea as with the assets: We put the subroutines into their own file/module and then create a header so other files/modules know where to find the subroutines. While ca65's smart mode is pretty good, it sometimes can go astray, especially when you're code is spread among several files. So in lines 15 and 16, I tell the assembler explicitly which register sizes to assume. Mind that this does not generate any code. The assembler will still track `rep` and `sep` instructions but will need some help from time to time as your project's code grows.

You should have a good understanding of it now, so I'm going to skip the necessary updates to the CMake files. If it gets confusing, you can see the final code on Github.

As a final step, we'll move the `VECTOR` segment into its own file `src/common/Vector.s`, as well as create a separate `src/Game/Init.s` file:

```
1  .ifndef INIT_INC
2  INIT_INC = 1
3  ; ---------------------------------------------------------------------------
4  ;   File: Init.inc
5  ;   Description: Holds address symbols for all subroutines defined in Init.s
6  ; ---------------------------------------------------------------------------
7
8  ;----- Import ----------------------------------------------------------------
```

```
.import     InitDemo
;------------------------------------------------------------------------------
```

Init.inc hosted with ❤ by **GitHub** **view raw**

```
; ------------------------------------------------------------------------------
;   File: Init.s
;   Description: Holds subroutines to initialize the demo
; ------------------------------------------------------------------------------

;----- Export ----------------------------------------------------------------
.export     InitDemo
;------------------------------------------------------------------------------

;----- Assembler Directives --------------------------------------------------
.p816                           ; tell the assembler this is 65816 code
.a8
.i16
;------------------------------------------------------------------------------

;----- Includes --------------------------------------------------------------
.include "Assets.inc"
.include "GameConstants.inc"
.include "MemoryMapWRAM.inc"
.include "PPU.inc"
.include "Registers.inc"
;------------------------------------------------------------------------------

.segment "CODE"
;------------------------------------------------------------------------------
;   This initializes the demo
;------------------------------------------------------------------------------
.proc   InitDemo
        ; load sprites into VRAM
        tsx                     ; save current stack pointer
        pea $0000               ; push VRAM destination address to stack
        pea SpriteData          ; push sprite data source address to stack
        lda #$80                ; number of bytes (128/$80) to transfer
        pha
        jsr LoadVRAM            ; transfer sprite data to VRAM
        txs                     ; restore old stack pointer

        ; load color data into CGRAM
        tsx                     ; save current stack pointer
```

```
40          lda #$80                ; destination address in CGRAM
41          pha
42          pea ColorData           ; color data source address
43          lda #$20                ; number of bytes (32/$20) to transfer
44          pha
45          jsr LoadCGRAM           ; transfer color data into CGRAM
46          txs                     ; restore old stack pointer
47
48          ; initialize OAMRAM mirror
49          ldx #$00
50          ; upper-left sprite
51          lda #(SCREEN_RIGHT/2 - SPRITE_SIZE) ; sprite 1, horizontal position
52          sta OAMMIRROR, X
53          inx                             ; increment index
54          lda #(SCREEN_BOTTOM/2 - SPRITE_SIZE); sprite 1, vertical position
55          sta OAMMIRROR, X
56          inx
57          lda #$00                        ; sprite 1, name
58          sta OAMMIRROR, X
59          inx
60          lda #$00                        ; no flip, palette 0
61          sta OAMMIRROR, X
62          inx
63          ; upper-right sprite
64          lda #(SCREEN_RIGHT/2)           ; sprite 3, horizontal position
65          sta OAMMIRROR, X
66          inx                             ; increment index
67          lda #(SCREEN_BOTTOM/2 - SPRITE_SIZE); sprite 3, vertical position
68          sta OAMMIRROR, X
69          inx
70          lda #$01                        ; sprite 3, name
71          sta OAMMIRROR, X
72          inx
73          lda #$00                        ; no flip, palette 0
74          sta OAMMIRROR, X
75          inx
76          ; lower-left sprite
77          lda #(SCREEN_RIGHT/2 - SPRITE_SIZE) ; sprite 2, horizontal position
78          sta OAMMIRROR, X
79          inx                             ; increment index
80          lda #(SCREEN_BOTTOM/2)          ; sprite 2, vertical position
81          sta OAMMIRROR, X
82          inx
83          lda #$02                        ; sprite 2, name
```

```
84          sta OAMMIRROR, X
85          inx
86          lda #$00                        ; no flip, palette 0
87          sta OAMMIRROR, X
88          inx
89          ; lower-right sprite
90          lda #(SCREEN_RIGHT/2)            ; sprite 4, horizontal position
91          sta OAMMIRROR, X
92          inx                             ; increment index
93          lda #(SCREEN_BOTTOM/2)          ; sprite 4, vertical position
94          sta OAMMIRROR, X
95          inx
96          lda #$03                        ; sprite 4, name
97          sta OAMMIRROR, X
98          inx
99          lda #$00                        ; no flip, palette 0
100         sta OAMMIRROR, X
101         inx
102         ; move the other sprites off screen
103         rep #$20                        ; set A to 16-bit
104         lda #$f180                      ; Y = 241, X = -128
105 OAMLoop:
106         sta OAMMIRROR, X
107         inx
108         inx
109         cpx #(OAMMIRROR_SIZE - $20)
110         bne OAMLoop
111         ; correct bit 9 of horizontal/X position, set size to 8x8
112         lda #$5555
113 OBJLoop:
114         sta OAMMIRROR, X
115         inx
116         inx
117         cpx #OAMMIRROR_SIZE
118         bne OBJLoop
119
120         sep #$20                        ; set A to 8-bit
121         ; correct extra OAM byte for first four sprites
122         ldx #$0200
123         lda #$00
124         sta OAMMIRROR, X
125
126         ; set initial horizontal and vertical speed
127         lda #SPRITE_SPEED
```

```
128         sta HOR_SPEED
129         sta VER_SPEED
130
131         ; make Objects visible
132         lda #$10
133         sta TM
134         ; release forced blanking, set screen to full brightness
135         lda #$0f
136         sta INIDISP
137         ; enable NMI, turn on automatic joypad polling
138         lda #$81
139         sta NMITIMEN
140
141         ; jmp GameLoop           ; all initialization is done
142         rts                     ; all initialization is done
143 .endproc
```

Init.s hosted with ❤ by **GitHub** **view raw**

```
1  ; ------------------------------------------------------------------------
2  ;   File: Vector.s
3  ;   Description: Interrup and reset vectors for the 65816 CPU
4  ; ------------------------------------------------------------------------
5
6  ;----- Assembler Directives ---------------------------------------------
7  .p816                        ; tell the assembler this is 65816 code
8  ;------------------------------------------------------------------------
9
10 ;----- Imports ----------------------------------------------------------
11 .import     NMIHandler         ; We need these two addresses so the SNES ...
12 .import     ResetHandler       ; ... knows where you start execution
13 ;------------------------------------------------------------------------
14
15 .segment "VECTOR"
16 ;------------------------------------------------------------------------
17 ; native mode   COP,         BRK,         ABT,
18 .addr           $0000,       $0000,       $0000
19 ;               NMI,         RST,         IRQ
20 .addr           NMIHandler,  $0000,       $0000
21
22 .word           $0000, $0000    ; four unused bytes
23
24 ; emulation m.  COP,         BRK,         ABT,
25 .addr           $0000,       $0000,       $0000
```

```
;               NMI,        RST,        IRQ
.addr           $0000,      ResetHandler, $0000
```

Vector.s hosted with ❤ by **GitHub** **view raw**

The only thing left to do, is to refactor your `src/JoypadSprite.s`:

```
; ------------------------------------------------------------------------
;   File: JoypadSprite.s
;   Description: Displays a sprite that bounces off the edges
; ------------------------------------------------------------------------

;----- Export ------------------------------------------------------------
.export     ResetHandler        ; export the entry point of the demo
.export     NMIHandler
;------------------------------------------------------------------------

;----- Assembler Directives ----------------------------------------------
.p816                           ; tell the assembler this is 65816 code
;------------------------------------------------------------------------

;----- Includes ----------------------------------------------------------
.include "Assets.inc"
.include "GameConstants.inc"
.include "Init.inc"
.include "MemoryMapWRAM.inc"
.include "PPU.inc"
.include "Registers.inc"
;------------------------------------------------------------------------

.segment "CODE"
;------------------------------------------------------------------------
;   This is the entry point of the demo
;------------------------------------------------------------------------
.proc   ResetHandler
        sei                     ; disable interrupts
        clc                     ; clear the carry flag
        xce                     ; switch the 65816 to native (16-bit mode)
        rep #$10                ; set X and Y to 16-bit
        sep #$20                ; set A to 8-bit
        lda #$8f                ; force v-blanking
        sta INIDISP
        stz NMITIMEN            ; disable NMI
        ; set the stack pointer to $1fff
```

```
38          ldx #$1fff              ; load X with $1fff
39          txs                     ; copy X to stack pointer
40          jsr InitDemo            ; initialize the demo
41          jmp GameLoop            ; enter main game loop
42  .endproc
43  ;-------------------------------------------------------------------------
44
45  ;-------------------------------------------------------------------------
46  ;   Executed during V-Blank
47  ;-------------------------------------------------------------------------
48  .proc   GameLoop
49          wai                                 ; wait for NMI / V-Blank
50
51          ; read joypad 1
52          ; check whether joypad is ready
53  WaitForJoypad:
54          lda HVBJOY                          ; get joypad status
55          and #$01                            ; check whether joypad done readin
56          beq WaitForJoypad                   ; ...if not, wait a bit more
57          ; first, check for newly pressed buttons since last frame
58          rep #$20                            ; set A to 16-bit
59          lda STDCNTRL1L                      ; get new input from this frame
60          ldy JOYPAD1                         ; get input from last frame
61          sta JOYPAD1                         ; store new input from this frame
62          tya                                 ; check for newly pressed buttons.
63          eor JOYPAD1                         ; filter buttons that were not pre
64          and JOYPAD1                         ; filter held buttons from last fr
65          sta JOYTRIGGER1                     ; ...and store them
66          ; second, check for buttons held from last frame
67          tya                                 ; get input from last frame
68          and JOYPAD1                         ; filter held buttons from last fr
69          sta JOYHELD1                        ; ...store them
70
71          ; check the dpad, if any of the directional buttons where pressed or h
72          ; move the sprites accordingly
73  CheckUpButton:
74          lda #$0000                          ; set A to zero
75          ora JOYTRIGGER1                     ; check whether the up button was
76          ora JOYHELD1                        ; ...or held from last frame
77          and #UP_BUTTON
78          beq CheckUpButtonDone               ; if neither has occured, move on
79          ; else, move sprites up
80          ldy #$0000                          ; Y is the loop counter
81          ldx #$0001                          ; set offset to 1, to manipulate s
```

```
82          sep #$20                            ; set A to 8-bit
83  MoveSpritesUp:
84          lda OAMMIRROR, X
85          sec
86          sbc VER_SPEED
87          bcc CorrectVerticalPositionDown     ; if vertical position is below ze
88          sta OAMMIRROR, X
89          inx                                 ; increment X by 4
90          inx
91          inx
92          inx
93          iny
94          cpy #$0004                          ; unless Y = 4, continue loop
95          bne MoveSpritesUp
96  CheckUpButtonDone:
97          rep #$20                            ; set A to 16-bit
98
99  CheckDownButton:
100         lda #$0000                          ; set A to zero
101         ora JOYTRIGGER1                     ; check whether the down button wa
102         ora JOYHELD1                        ; ...or held from last frame
103         and #DOWN_BUTTON
104         beq CheckDownButtonDone             ; if neither has occured, move on
105         ; else, move sprites down
106         ldy #$0000                          ; Y is the loop counter
107         ldx #$0001                          ; set offset to 1, to manipulate s
108         sep #$20                            ; set A to 8-bit
109         ; check if sprites move below buttom boundry
110         lda OAMMIRROR, X
111         clc
112         adc VER_SPEED
113         cmp #(SCREEN_BOTTOM - 2 * SPRITE_SIZE)
114         bcs CorrectVerticalPositionUp
115 MoveSpritesDown:
116         lda OAMMIRROR, X
117         clc
118         adc VER_SPEED
119         sta OAMMIRROR, X
120         inx                                 ; increment X by 4
121         inx
122         inx
123         inx
124         iny
125         cpy #$0004                          ; unless Y = 4, continue loop
```

```
126         bne MoveSpritesDown
127 CheckDownButtonDone:
128         rep #$20                        ; set A to 16-bit
129         jmp CheckLeftButton             ; continue input check
130
131 CorrectVerticalPositionDown:
132         sep #$20                        ; set A to 8-bit
133         stz OAMMIRROR + 1               ; sprite 1, vertical position
134         stz OAMMIRROR + 5               ; sprite 3, vertical position
135         lda #SPRITE_SIZE
136         sta OAMMIRROR + 9               ; sprite 2, vertical position
137         sta OAMMIRROR + 13              ; sprite 4, vertical position
138         rep #$20                        ; set A to 16-bit
139         jmp CheckLeftButton             ; continue input check
140
141 CorrectVerticalPositionUp:
142         sep #$20                        ; set A to 8-bit
143         lda #(SCREEN_BOTTOM - 2 * SPRITE_SIZE)
144         sta OAMMIRROR + 1               ; sprite 1, vertical position
145         sta OAMMIRROR + 5               ; sprite 3, vertical position
146         lda #(SCREEN_BOTTOM - SPRITE_SIZE)
147         sta OAMMIRROR + 9               ; sprite 2, vertical position
148         sta OAMMIRROR + 13              ; sprite 4, vertical position
149         rep #$20                        ; set A to 16-bit
150
151 CheckLeftButton:
152         lda #$0000                      ; set A to zero
153         ora JOYTRIGGER1                 ; check whether the up button was
154         ora JOYHELD1                    ; ...or held from last frame
155         and #LEFT_BUTTON
156         beq CheckLeftButtonDone         ; if neither has occured, move on
157         ; else, move sprites up
158         ldy #$0000                      ; Y is the loop counter
159         ldx #$0000                      ; set offset to 0, to manipulate s
160         sep #$20                        ; set A to 8-bit
161 MoveSpritesLeft:
162         lda OAMMIRROR, X
163         sec
164         sbc HOR_SPEED
165         bcc CorrectHorizontalPositionRight
166         sta OAMMIRROR, X
167         inx                             ; increment X by 4
168         inx
169         inx
```

```
170         inx
171         iny
172         cpy #$0004                      ; unless Y = 4, continue loop
173         bne MoveSpritesLeft
174 CheckLeftButtonDone:
175         rep #$20                        ; set A to 16-bit
176
177 CheckRightButton:
178         lda #$0000                      ; set A to zero
179         ora JOYTRIGGER1                 ; check whether the down button wa
180         ora JOYHELD1                    ; ...or held from last frame
181         and #RIGHT_BUTTON
182         beq CheckRightButtonDone        ; if neither has occured, move on
183         ; else, move sprites down
184         ldy #$0000                      ; Y is the loop counter
185         ldx #$0000                      ; set offset to 0, to manipulate s
186         sep #$20                        ; set A to 8-bit
187         ; check whether sprites move beyond right boundry
188         lda OAMMIRROR, X
189         clc
190         adc HOR_SPEED
191         cmp #(SCREEN_RIGHT - 2 * SPRITE_SIZE)
192         bcs CorrectHorizontalPositionLeft
193 MoveSpritesRight:
194         lda OAMMIRROR, X
195         clc
196         adc HOR_SPEED
197         sta OAMMIRROR, X
198         inx                             ; increment X by 4
199         inx
200         inx
201         inx
202         iny
203         cpy #$0004                      ; unless Y = 4, continue loop
204         bne MoveSpritesRight
205 CheckRightButtonDone:
206         rep #$20                        ; set A to 16-bit
207         jmp InputDone
208
209 CorrectHorizontalPositionRight:
210         sep #$20                        ; set A to 8-bit
211         stz OAMMIRROR + 0               ; sprite 1, horizontal position
212         stz OAMMIRROR + 8               ; sprite 2, horizontal position
213         lda #SPRITE_SIZE
```

```
214         sta OAMMIRROR + 4                   ; sprite 3, horizontal position
215         sta OAMMIRROR + 12                  ; sprite 4, horizontal position
216         jmp InputDone
217
218 CorrectHorizontalPositionLeft:
219         sep #$20                            ; set A to 8-bit
220         lda #(SCREEN_RIGHT - 2 * SPRITE_SIZE)
221         sta OAMMIRROR + 0                   ; sprite 1, horizontal position
222         sta OAMMIRROR + 8                   ; sprite 2, horizontal position
223         lda #(SCREEN_RIGHT - SPRITE_SIZE)
224         sta OAMMIRROR + 4                   ; sprite 3, horizontal position
225         sta OAMMIRROR + 12                  ; sprite 4, horizontal position
226
227 InputDone:
228         sep #$20                            ; set A back to 8-bit
229
230         jmp GameLoop
231 .endproc
232 ;-------------------------------------------------------------------------------
233
234 ;-------------------------------------------------------------------------------
235 ;   Will be called during V-Blank every frame
236 ;-------------------------------------------------------------------------------
237 .proc   NMIHandler
238         lda RDNMI               ; read NMI status, acknowledge NMI
239
240         tsx                     ; save old stack pointer
241         pea OAMMIRROR           ; push mirror address to stack
242         jsr UpdateOAMRAM        ; update OAMRAM
243         txs                     ; restore old stack pointer
244
245         rti
246 .endproc
247 ;-------------------------------------------------------------------------------
248
249 ;-------------------------------------------------------------------------------
250 ;   Is not used in this program
251 ;-------------------------------------------------------------------------------
252 .proc   IRQHandler
253         ; code
254         rti
255 .endproc
256 ;-------------------------------------------------------------------------------
```

JoypadSprite.s hosted with ❤ by **GitHub** **view raw**

Now it only contains your main game loop, which I think is much nicer to handle and expand this way.

Your final directory structure should now look like this:

```
snes_cradle/
    assets/
        Assets.inc
        Assets.s
        CMakeLists.txt
        SpriteColors.pal
        Sprites.vra
    cmake/
        CMakeCA65816Compiler.cmake.in
        CMakeCA65816Information.cmake
        CMakeDetermineCA65816Compiler.cmake
        CMakeTestCA65816Compiler.cmake
    src/
        common/
            CMakeLists.txt
            GameConstants.inc
            MemoryMapWRAM.inc
            Registers.inc
            Vector.s
        Game/
            CMakeLists.txt
            Init.inc
            Init.s
        PPU/
            CMakeLists.txt
            PPU.inc
            PPU.s
        CMakeLists.txt
        JoypadSprite.s
```

```
CMakeLists.txt
MemoryMap.cfg
```

Mind that is directory structure is pretty arbitrary and you can and should tailor it to your own needs. If you got lost somewhere, check out the full code here.

Conclusion

This and the last entry may seem to overly complicate things. But I'm certain that this will pay off in the long run. As of now, we have a simple project structure that can easily be expanded without it causing us headaches. There are a few things I've skipped for the sake of brevity. For example, it is possible to completely automate the generation of header files from sources with CMake. But I opted to keep things simple for now.

Next time, we'll finally return to more SNES programming. What's a game without backgrounds? Exactly, boring.

Thank you for reading and see you next time!

Links and References

- Another SNES project template by ARM9 that uses C
- The SNESKIT by Daniel Oaks includes very useful extra tools and code templates
- The full code of this tutorial

ALSO ON MACHINE CODE CONSTRUCTION YARD

SNES Assembly Adventure 11: ...
4 years ago · 2 comments
I do assembly programming on old machines and boat building.

SNES Assembly Adventure 06: More ...
8 years ago · 6 comments
I do assembly programming on old machines and boat building.

SNES Assembly Adventure 04: Display ...
8 years ago · 10 comments
I do assembly programming on old machines and boat building.

SNES Assem Adventure 0
8 years ago · 6 c
I do assembly on old machin building.